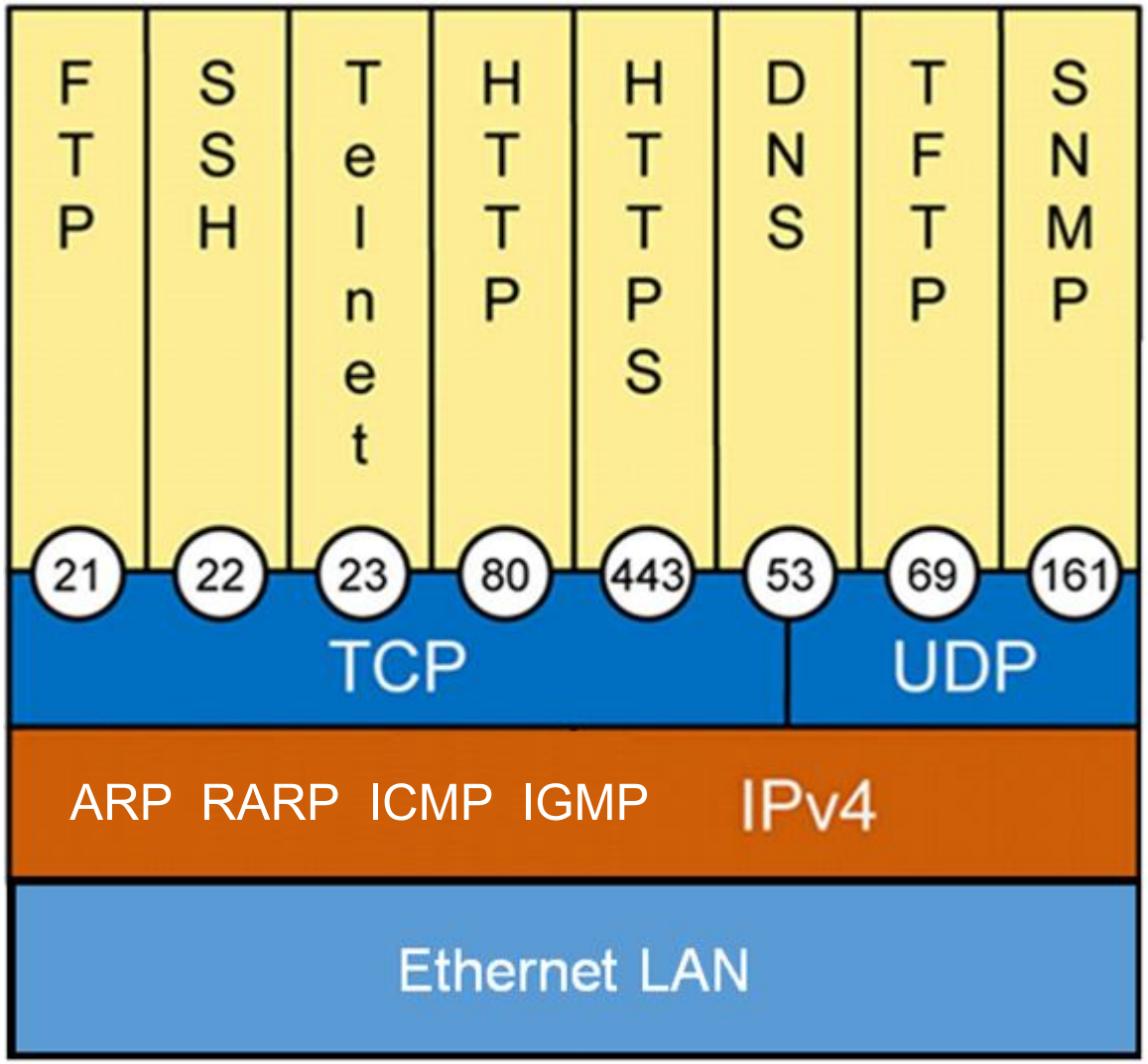
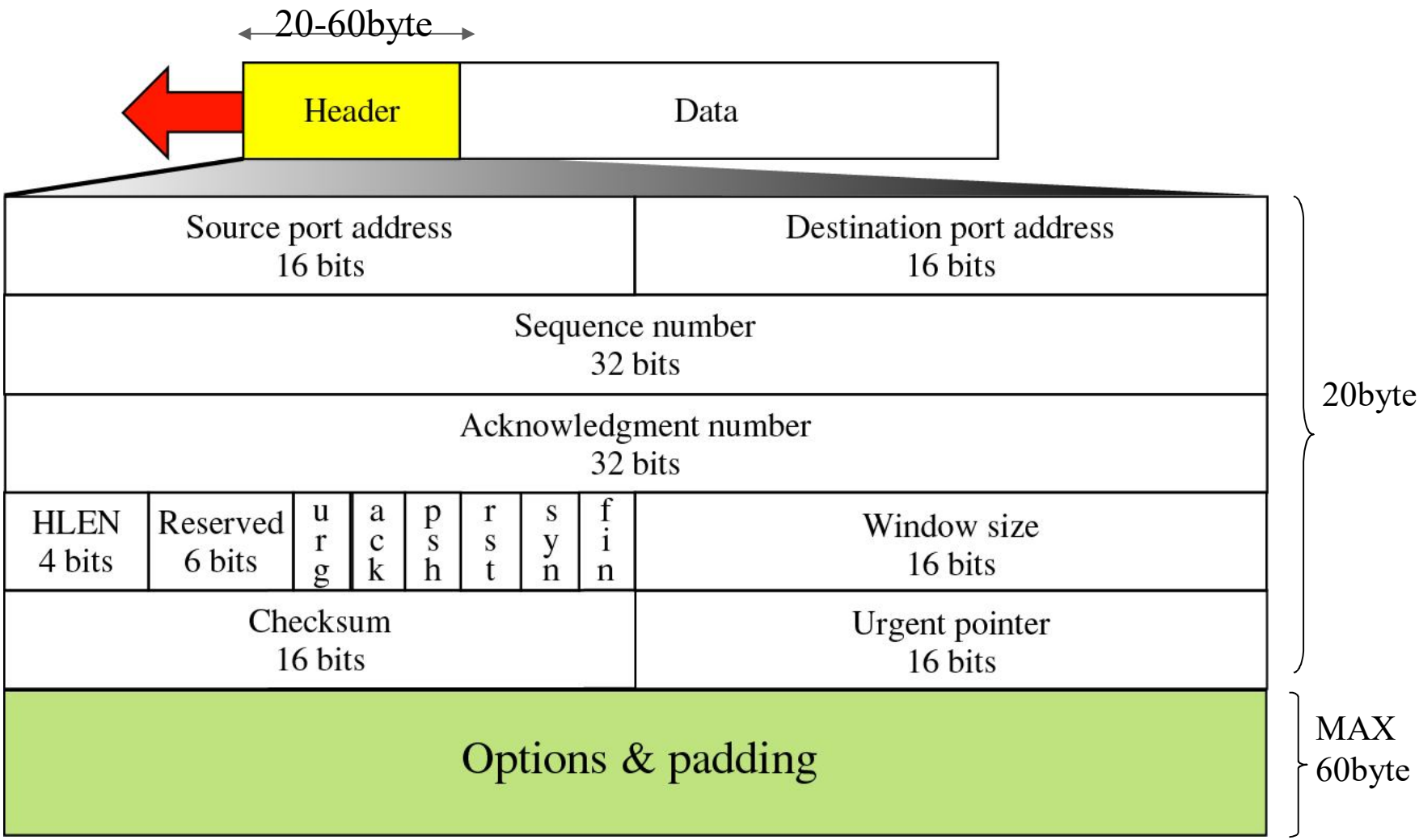
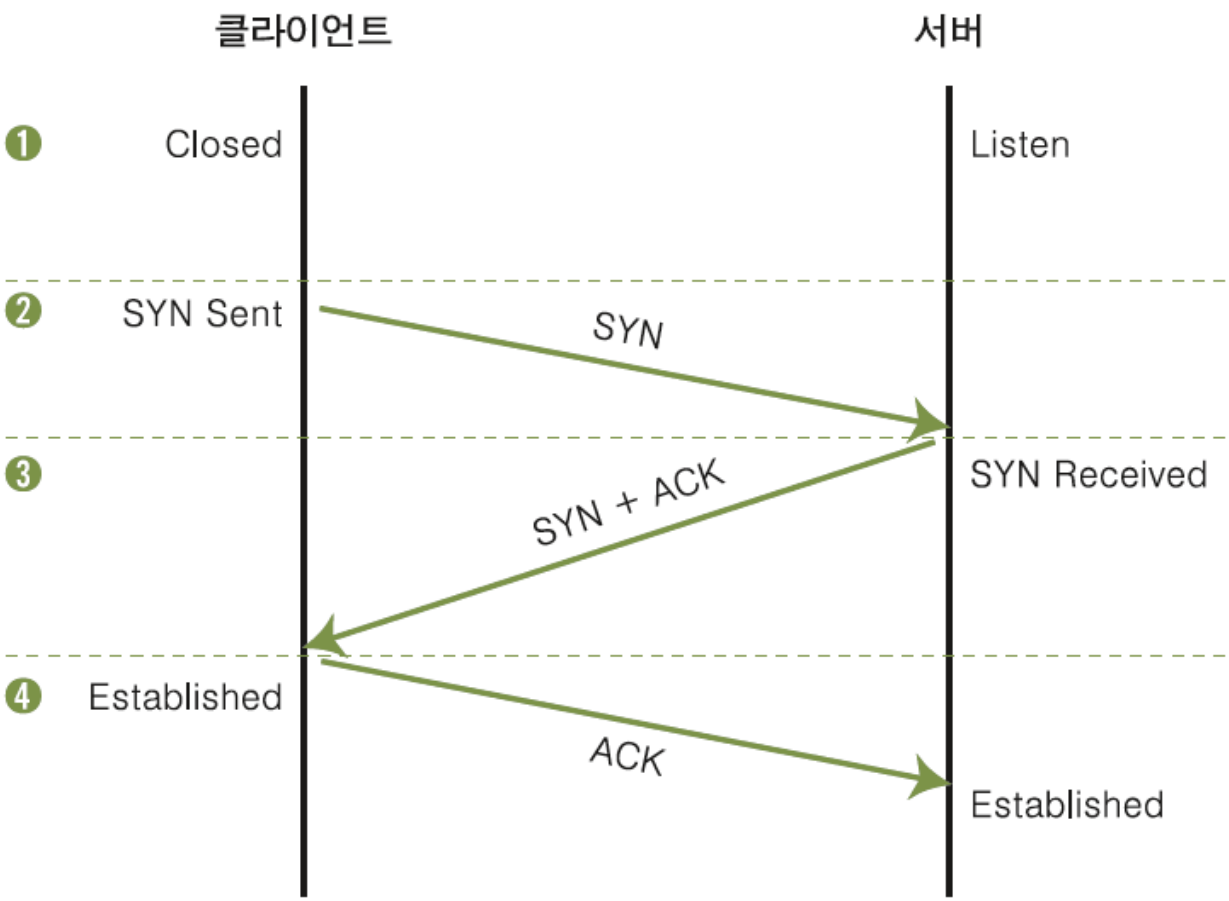
TCP/IP Protocol Stack



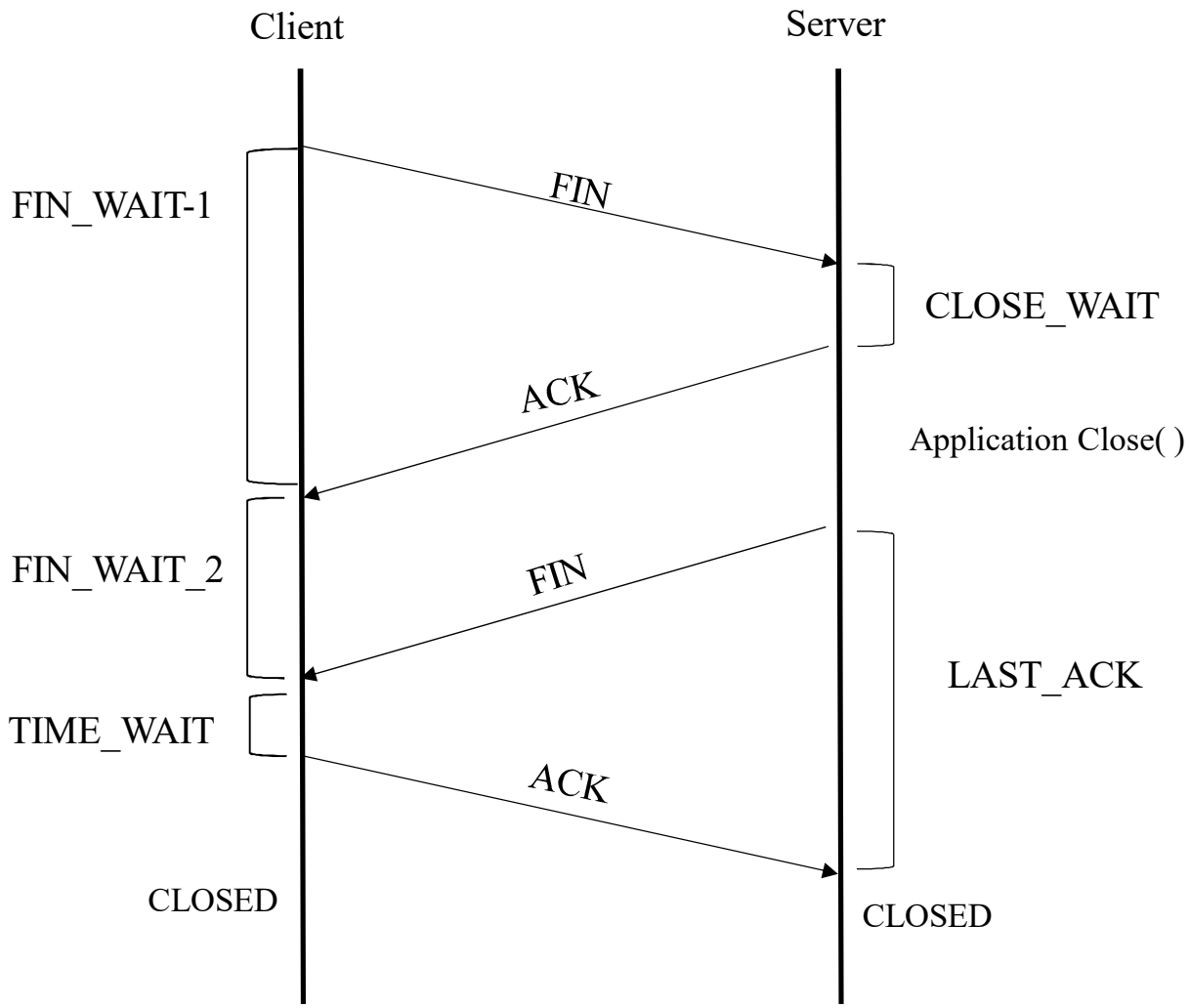
TCP Header


20-60byte
Header
Data
Source port address
16 bits
Destination port address
16 bits
Sequence number
32 bits
Acknowledgment number
32 bits
HLEN
4 bits
Reserved
6 bits
urg
ack
psh
rst
syn
fin
Window size
16 bits
Checksum
16 bits
Urgent pointer
16 bits
20byte
Options & padding
MAX
60byte

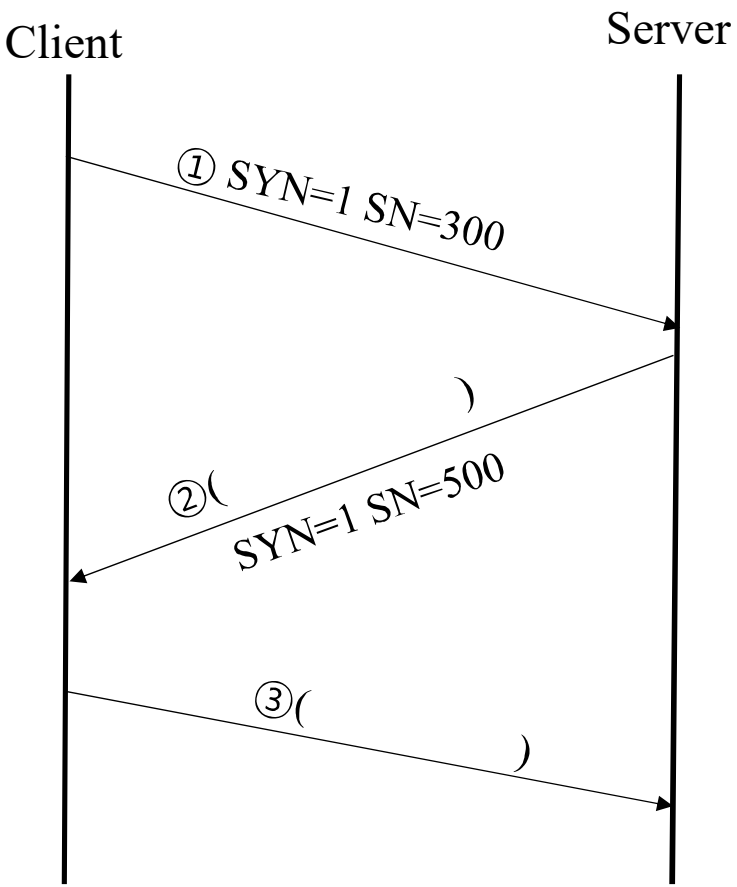
TCP 연결 설정 (3-Way Handshaking)

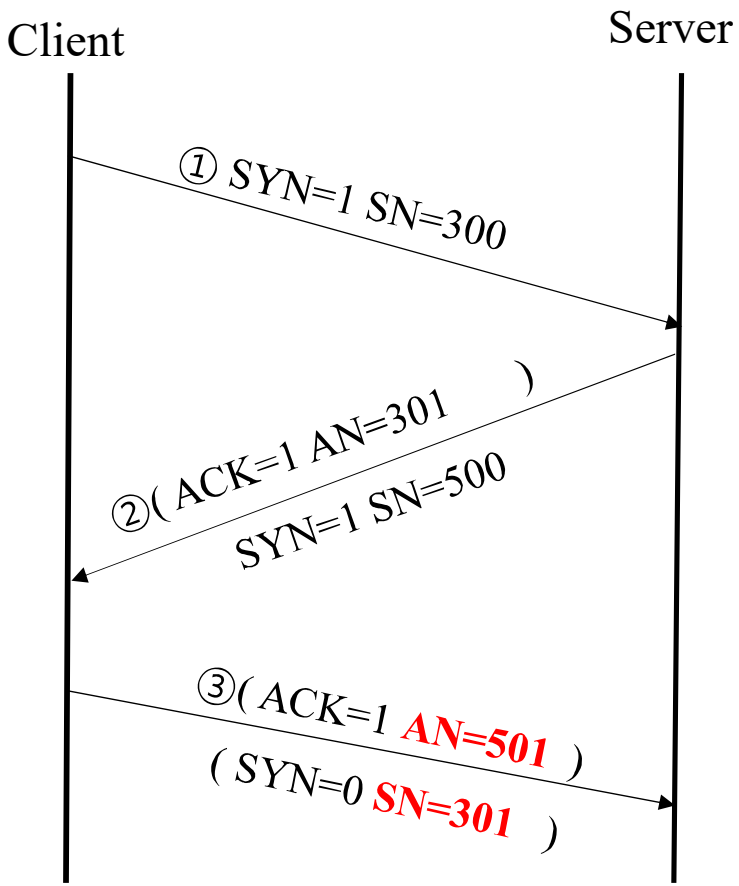


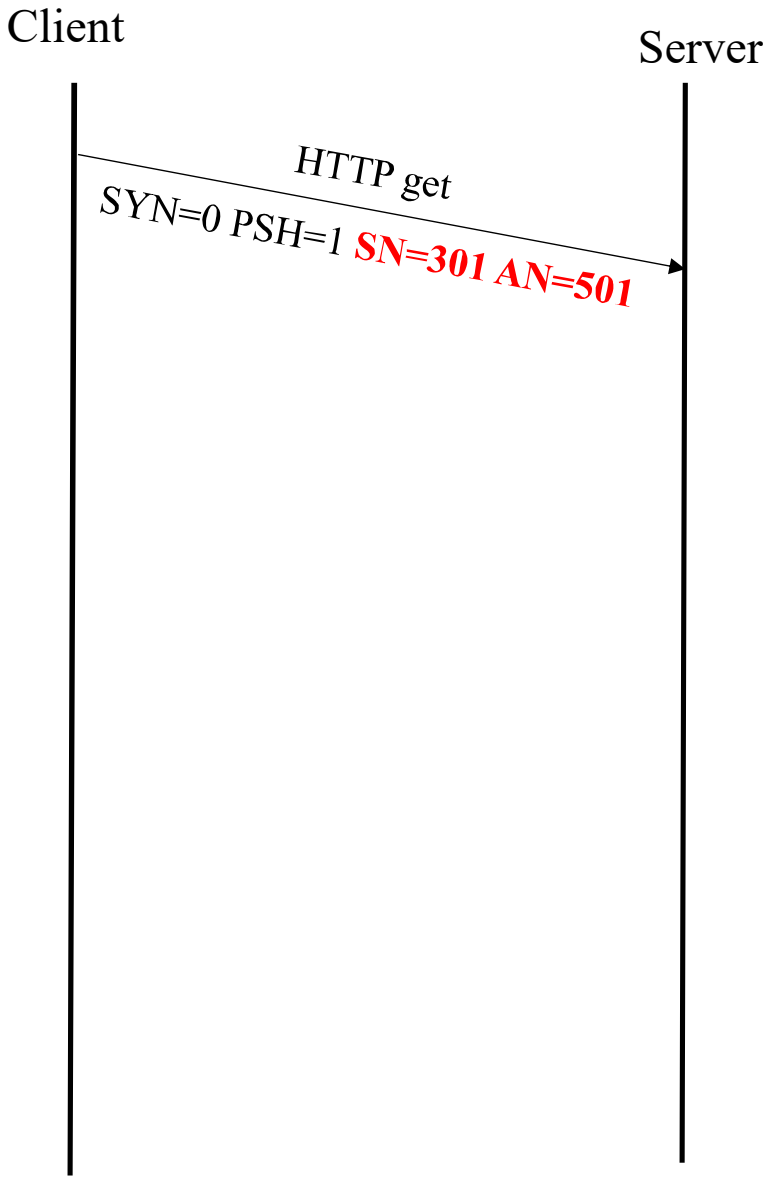
TCP 연결 종료(4-way Handshake)



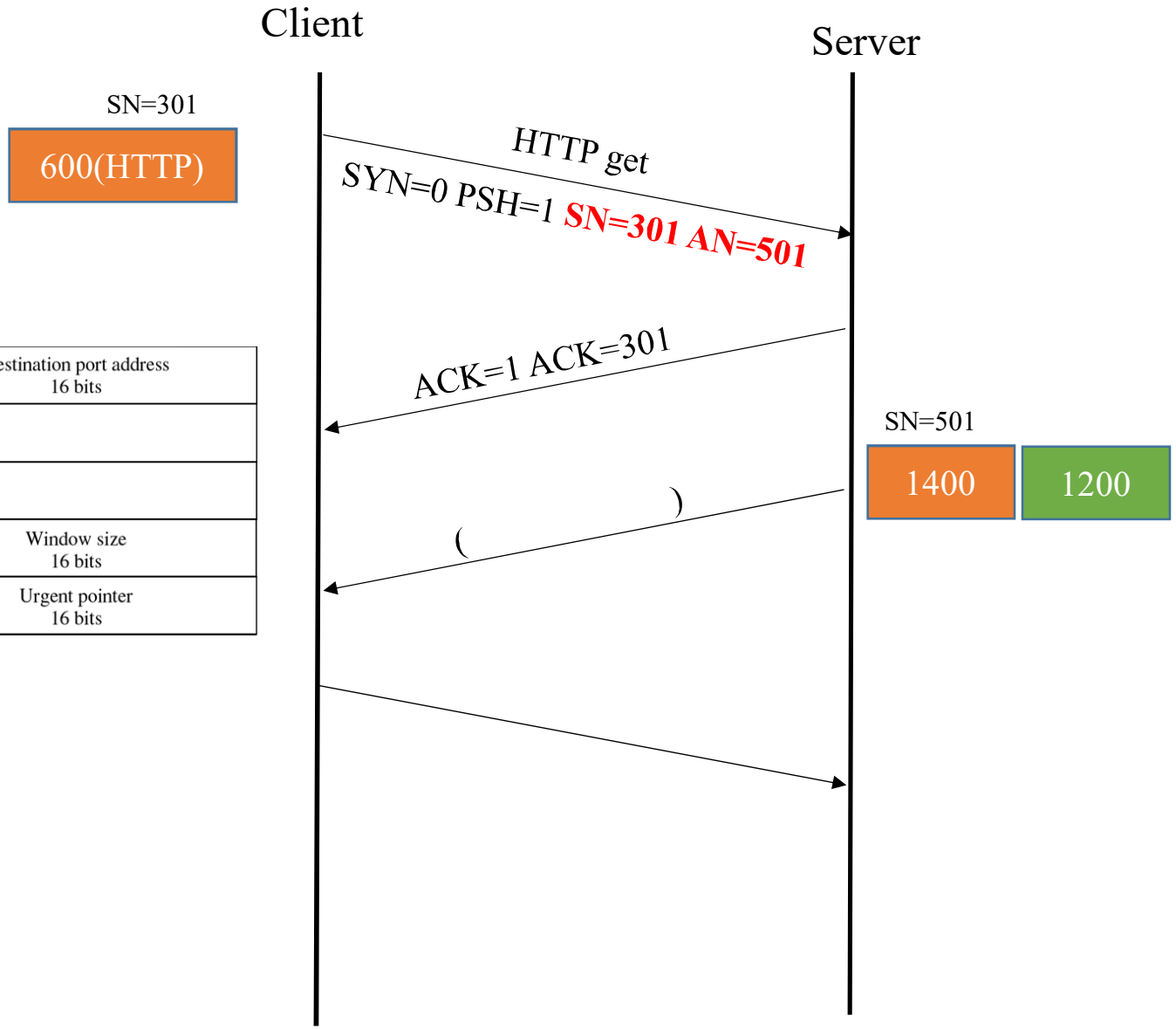
Source port address 16 bits								Destination port address 16 bits
Sequence number 32 bits								
Acknowledgment number 32 bits								
HLEN 4 bits	Reserved 6 bits	urg	ack	psh	rst	syn	fin	Window size 16 bits
Checksum 16 bits								Urgent pointer 16 bits



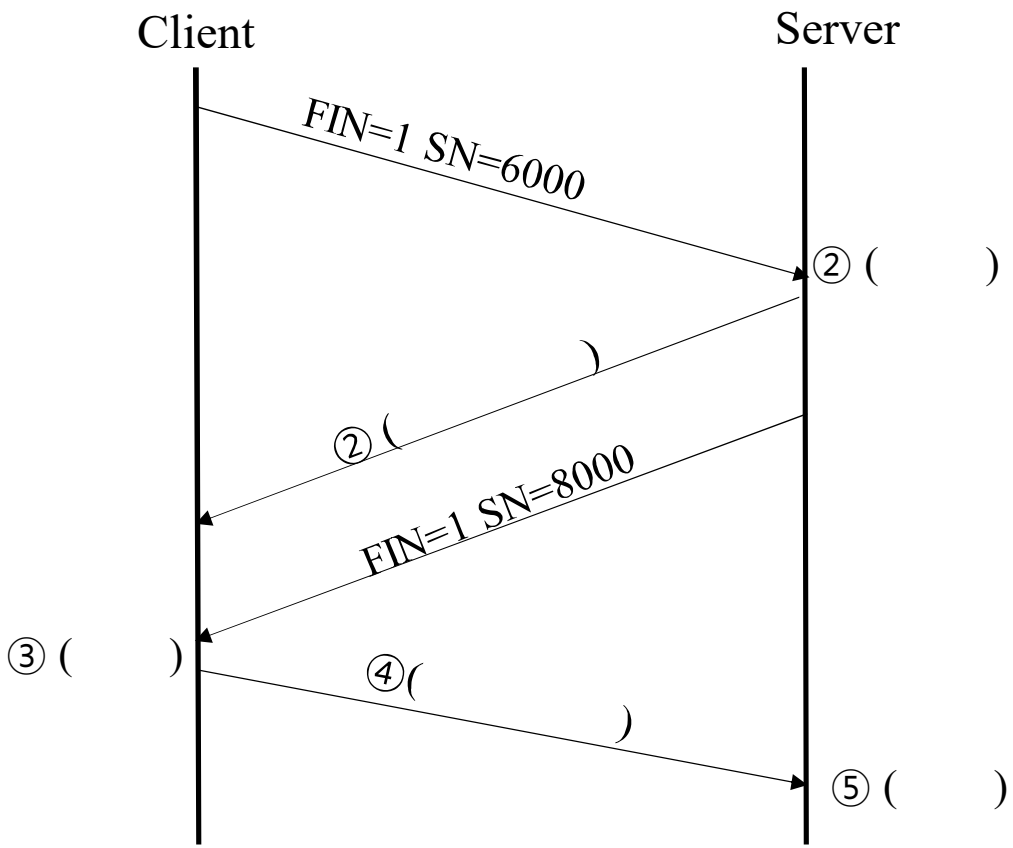

Client
Server
① SYN=1 SN=300
②(ACK=1 AN=301)
SYN=1 SN=500
③(ACK=1 AN=501)
(SYN=0 SN=301)


Client
Server
HTTP get
SYN=0 PSH=1 SN=301 AN=501

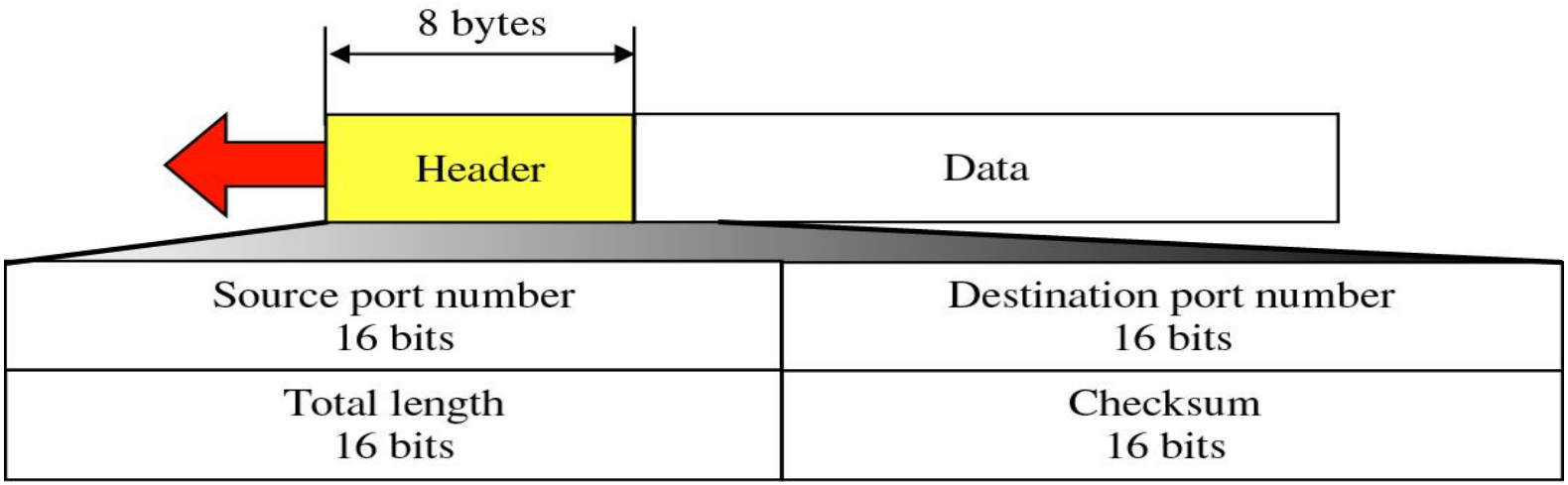
Source port address 16 bits								Destination port address 16 bits
Sequence number 32 bits								
Acknowledgment number 32 bits								
HLEN 4 bits	Reserved 6 bits	urg	ack	psh	rst	syn	fin	Window size 16 bits
Checksum 16 bits								Urgent pointer 16 bits



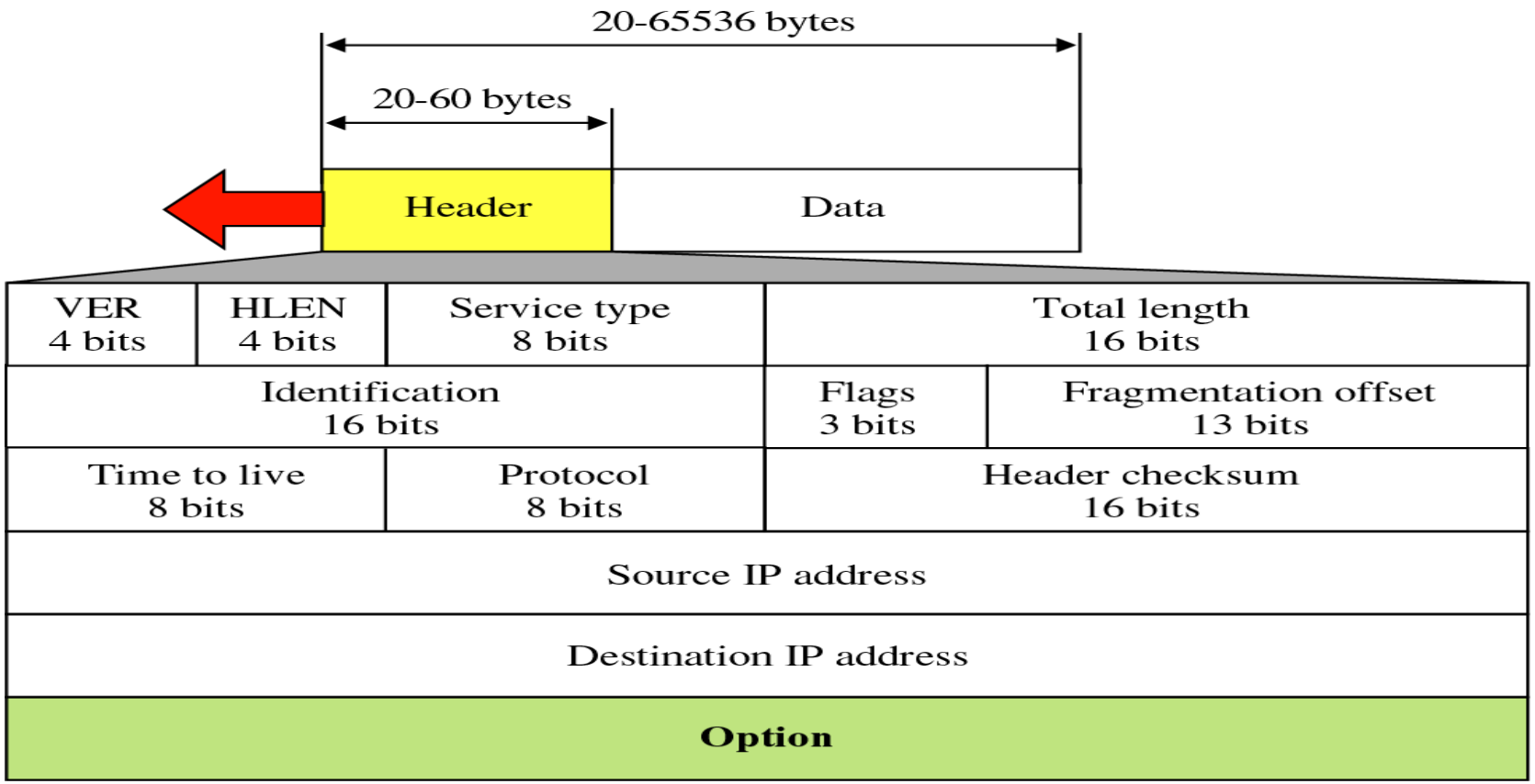
Client
Server
Window Size = 1500
SN=6100
SN=6900
SN=7500
SN=8000
SN=8700
800
600
500
700
200


Client
Server
FIN=1 SN=6000
② ( )
)
② (
FIN=1 SN=8000
③ ( )
④(
)
⑤ ( )

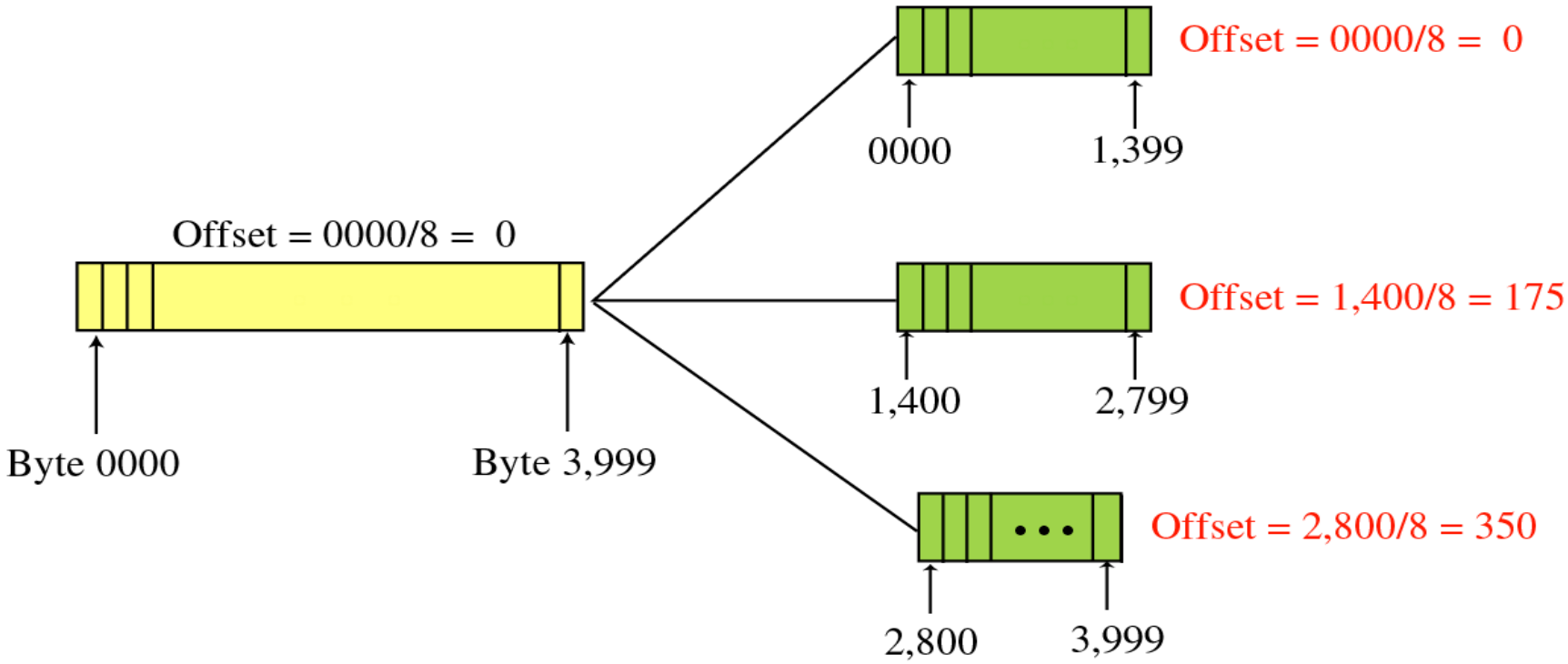
UDP Header


8 bytes
Header
Data
Source port number
16 bits
Destination port number
16 bits
Total length
16 bits
Checksum
16 bits

IP 헤더

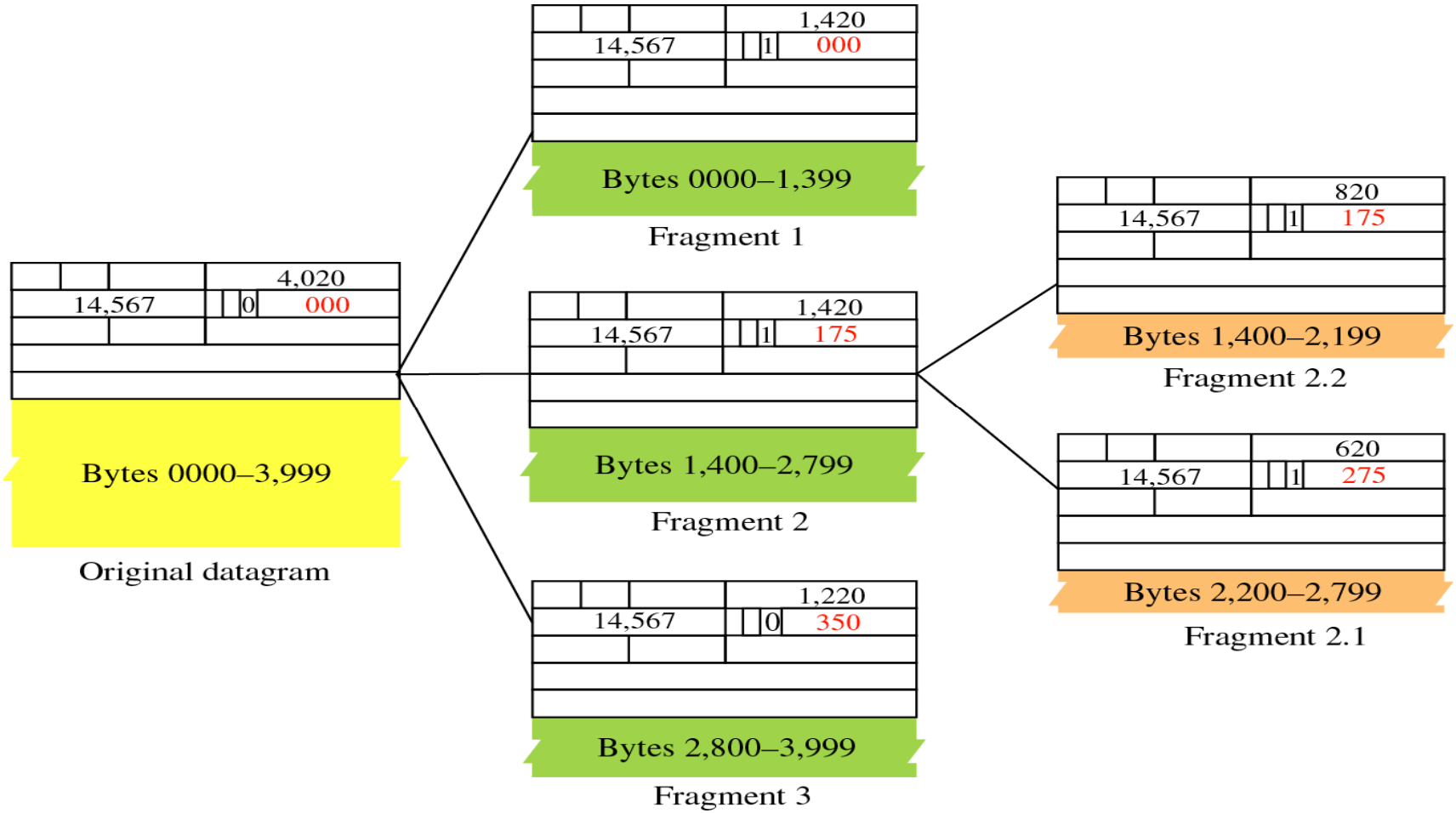

20-65536 bytes
20-60 bytes
Header
Data
VER
4 bits
HLEN
4 bits
Service type
8 bits
Total length
16 bits
Identification
16 bits
Flags
3 bits
Fragmentation offset
13 bits
Time to live
8 bits
Protocol
8 bits
Header checksum
16 bits
Source IP address
Destination IP address
Option

IP Datagram 분석 - Fragmentation



IP Datagram 분석 - Fragmentation

- Fragmentation이 다시 일어나는 경우
  - 두 번째 단편이 다시 800과 600으로 나누어지는 경우



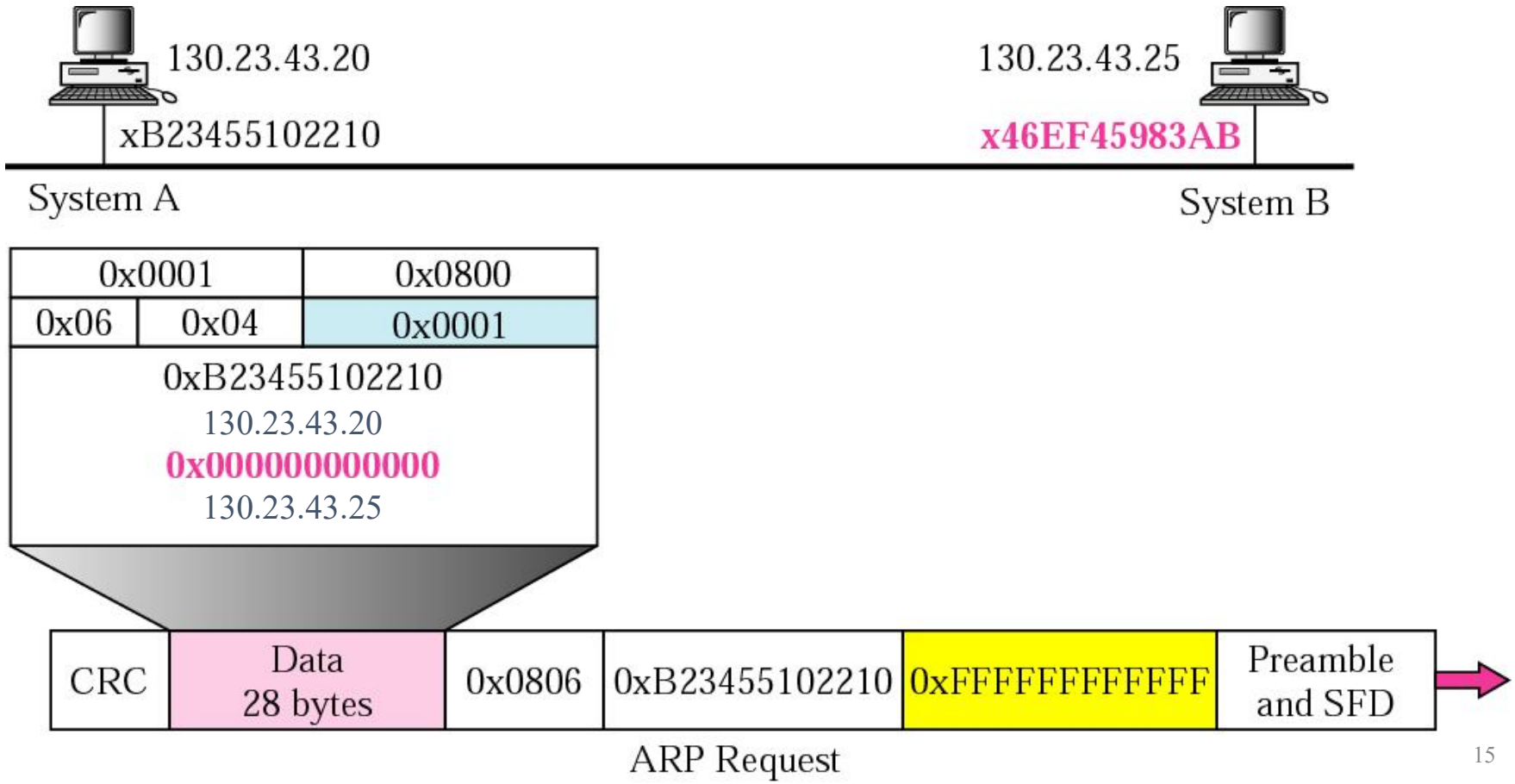
Ethernet Frame 구조

Preamble 56 bits of alternating 1s and 0s.
SFD Start field delimiter, flag (10101011)

Preamble	SFD	Destination address	Source address	Length PDU	Data and padding	CRC
7 bytes	1 byte	6 bytes	6 bytes	2 bytes		4 bytes

ARP Request Packet

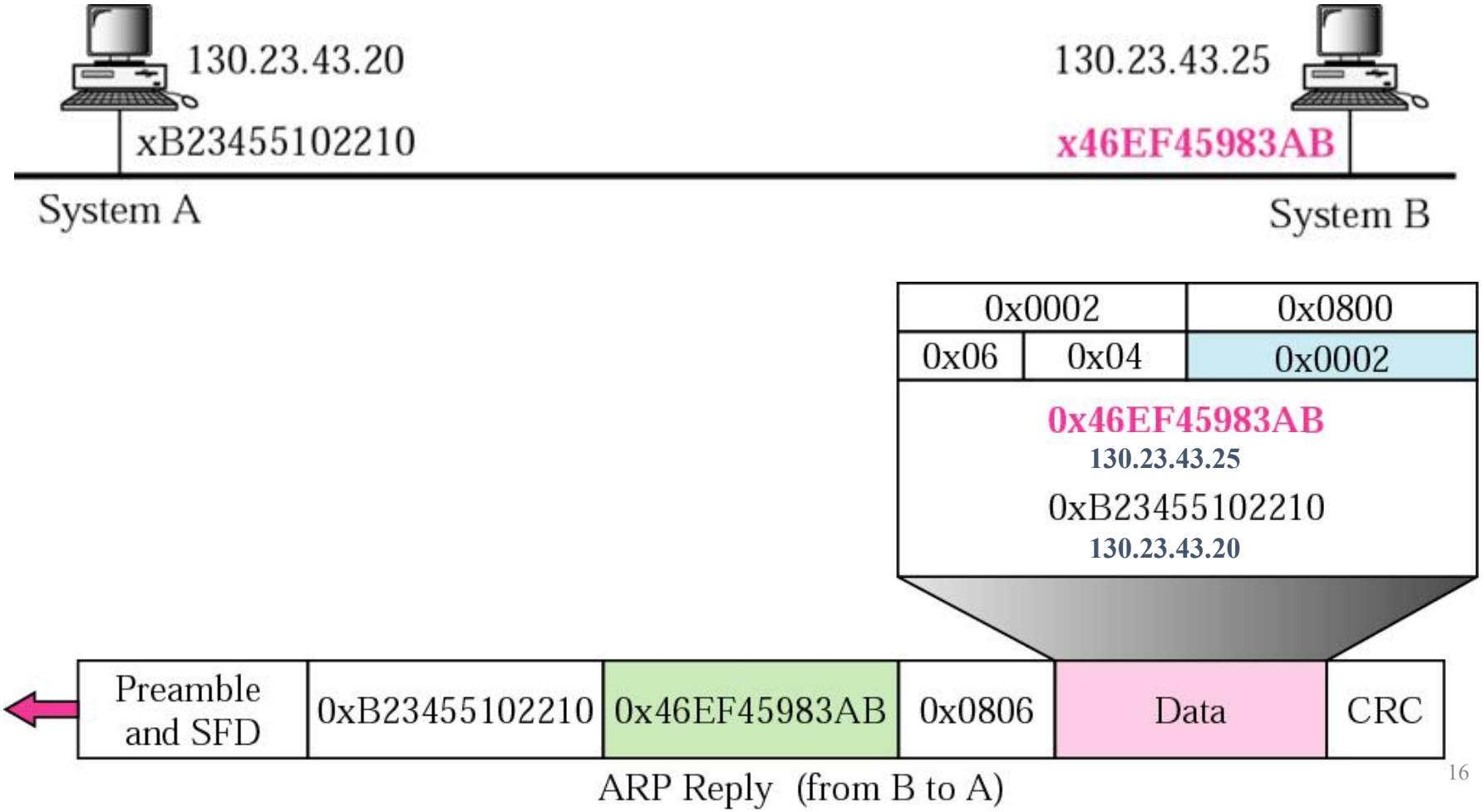
ARP Cache Table



ARP Reply Packet

ARP Cache Table

130.23.43.20 B234-5510-2210



ARP 수행 후 캐쉬 테이블

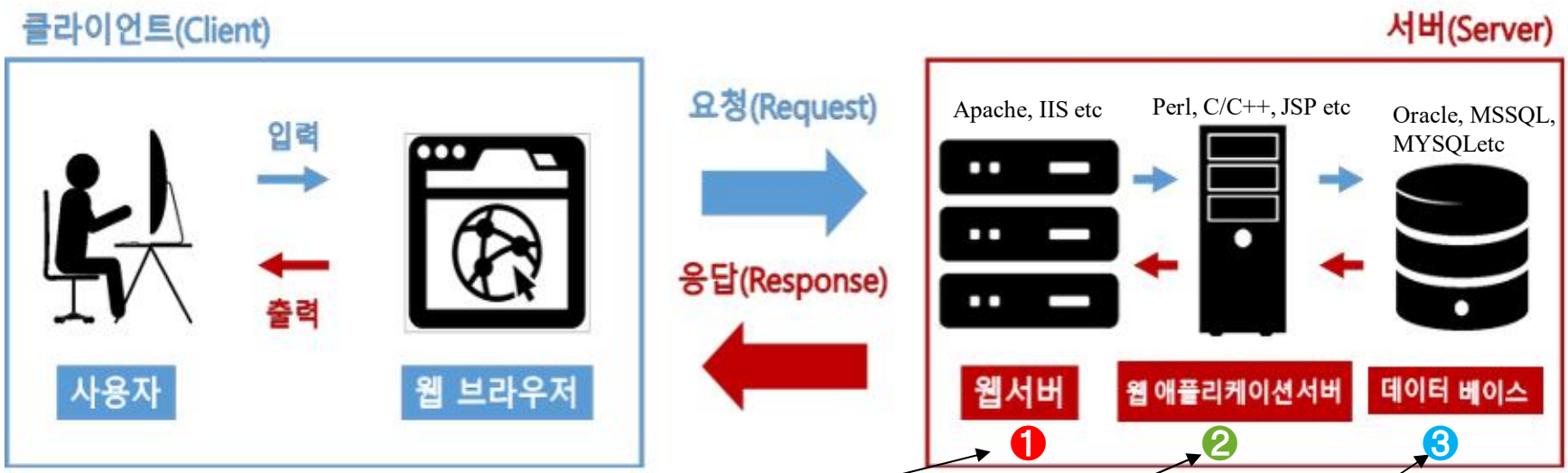
ARP Cache Table

130.23.43.25 46EF-4598-3AB

ARP Cache Table

130.23.43.20 B234-5510-2210

3-Tier Web System Architecture



http://www.google.co.kr:80/sport/guest.php?who=anony&type=human

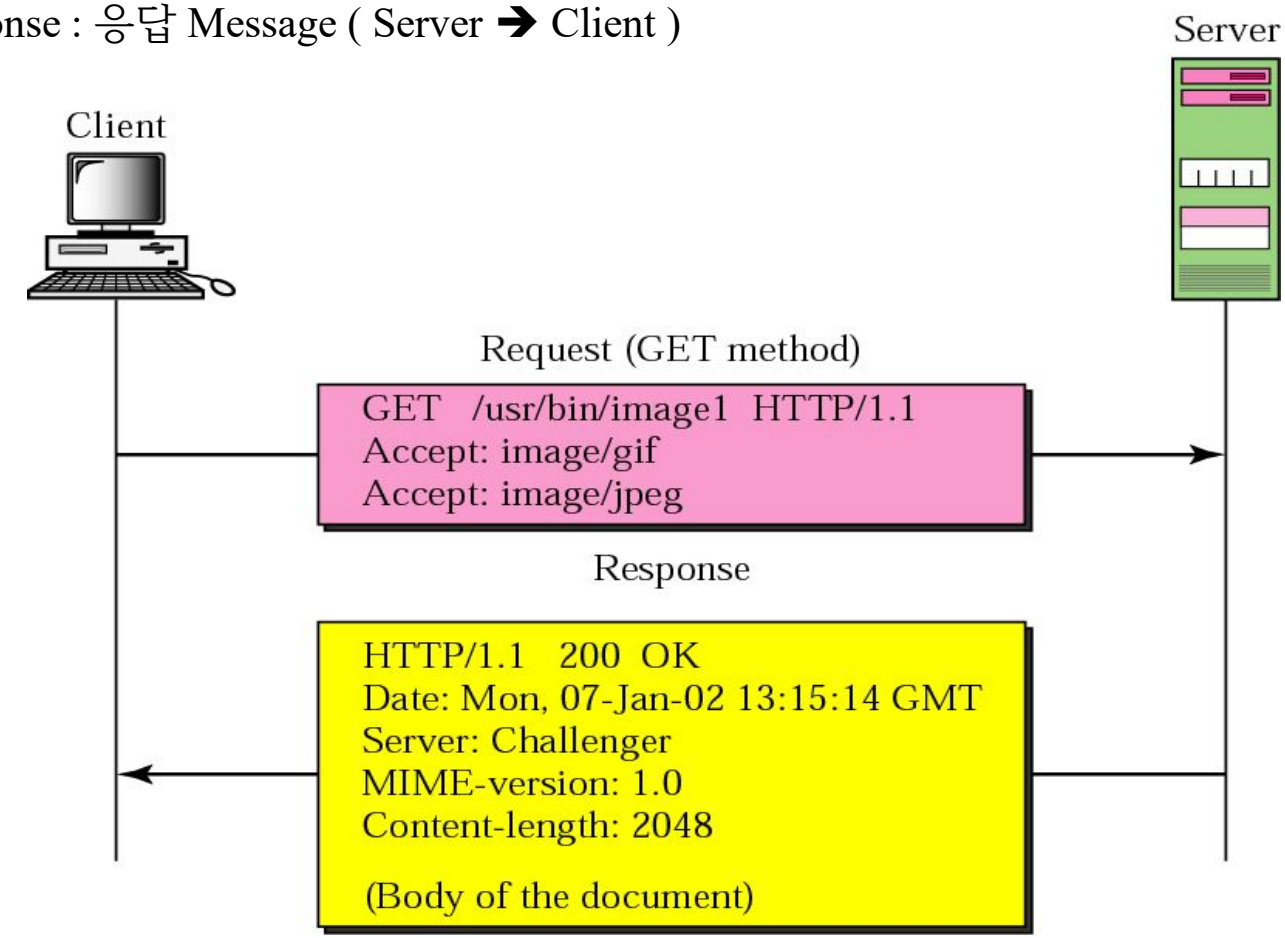
URI

프로토콜://주소(또는 IP):포트번호/경로 또는 파일?파라미터=값

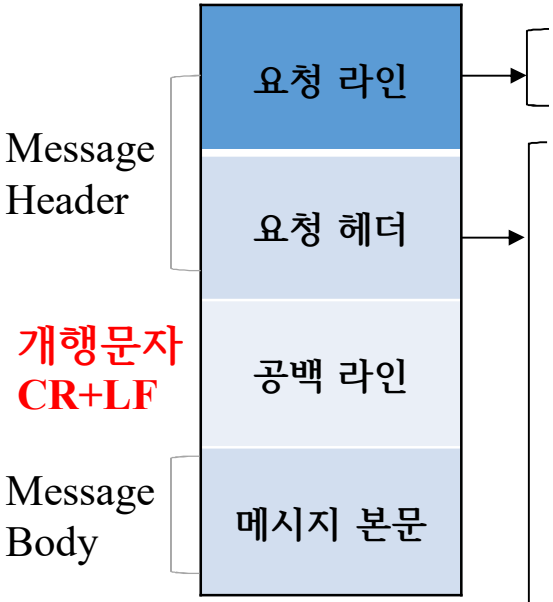
URL= 프로토콜://절대 주소
[FQDN][Port번호|PATH]

· Message Format

- Request : 요청 Message (Client ➔ Server)
- Response : 응답 Message (Server ➔ Client)



▪ HTTP Request Message




```
GET / HTTP/1.1
Accept: text/html, application/xhtml+xml, */*
Accept-Encoding: gzip, deflate
Cookie: HSID=AaxlkKoV2snlEi6UQ; SSID=AAYVu_evC0Tiu3aVc;
APISID=JEWp0eojRTLFtYKJ/ACgEWh_0mL8Li_-fl;
SAPISID=kabRBO-uT0ebDcfc/A2byDX--FwN649tHw
Host: www.google.com
Connection: Keep-Alive
Accept-Language: ko-KR
User-Agent: Mozilla/5.0 (compatible; MSIE 9.0; Windows NT 6.1; Trident/5.0)
```

*** CR+LF**
- CR(Carriage Return) : 16진수 0x0d
- LF(Line Feed) : 16진수 0x0a

2.2 HTTP Request Message Format

- HTTP Request

```
GET / HTTP/1.1
Accept: text/html, application/xhtml+xml, */*
Accept-Encoding: gzip, deflate
Cookie: HSID=AaxlkKoV2snlEi6UQ; SSID=AAYVu_evC0Tiu3aVc;
APISID=JEWp0eojRTLFtYKJ/ACgEWh_0mL8Li_-fl;
SAPISID=kabRBO-uT0ebDcfc/A2byDX--FwN649tHw
Host: www.google.com
Connection: Keep-Alive
Accept-Language: ko-KR
User-Agent: Mozilla/5.0 (compatible; MSIE 9.0; Windows NT 6.1; Trident/5.0)
```

전송방법/요청된 URL/HTTP 버전

데이터 타입

서버가 클라이언트에 전송한 인자 값에 추가 정보를 보낼 때 사용

URL 주소에 나타난 호스트 명을 자세하게 나타냄

브라우저나 기타 클라이언트의 소트프웨어 정보를 보여줌

■ Request Method 종류

- 서버에 임무를 부여하는 HTTP 메소드

- GET
- POST
- HEAD
- OPTIONS
- PUT
- DELETE
- TRACE

```
GET / HTTP/1.1
Accept: text/html, application/xhtml+xml, */*
Accept-Encoding: gzip, deflate
Cookie: HSID=AaxlkKoV2snlEi6UQ; SSID=AAYVu_evC0Tiu3aVc;
APISID=JEWp0eojRTLFtYKJ/ACgEWh_0mL8Li_-fl;
SAPISID=kabRBO-uT0ebDcfc/A2byDX--FwN649tHw
Host: www.google.com
Connection: Keep-Alive
Accept-Language: ko-KR
User-Agent: Mozilla/5.0 (compatible; MSIE 9.0; Windows NT 6.1; Trident/5.0)
```

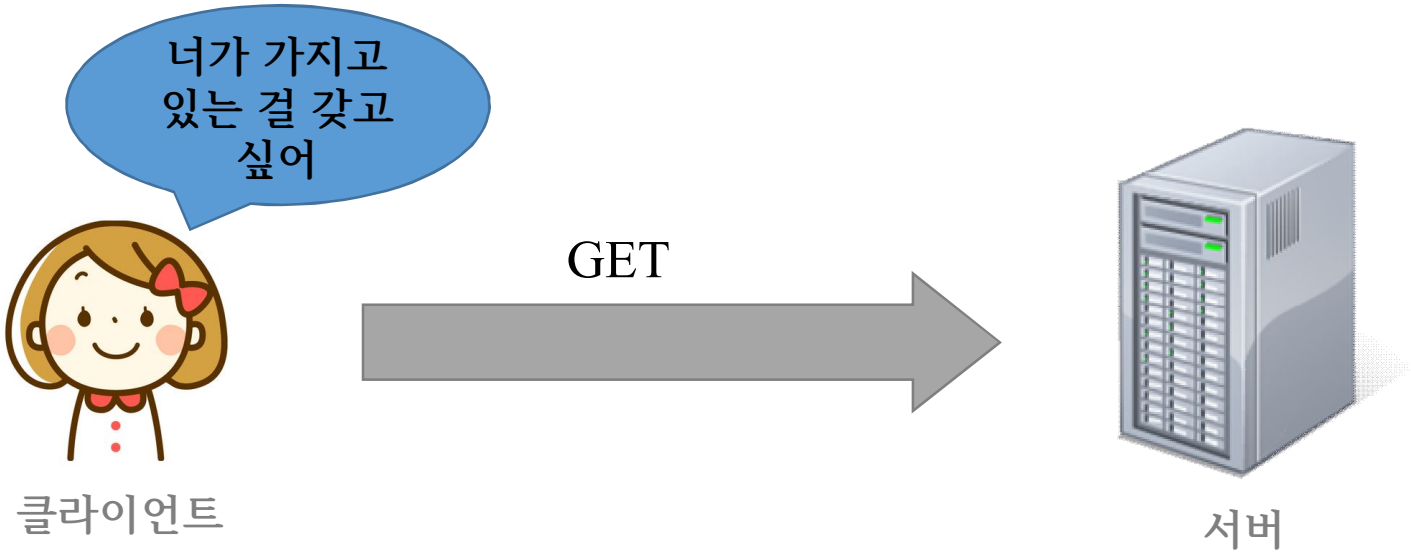
- HTTP Request

POST /form/entry HTTP/1.1

❶ 메소드 ❷ URI ❸ 프로토콜 버전

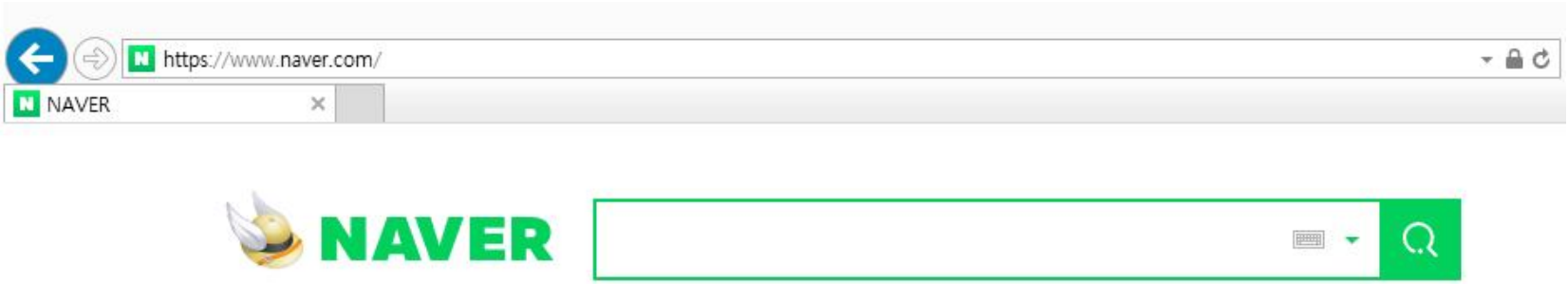
HTTP Request : Get 방식

- 가장 일반적인 HTTP Request 형태
- 요청 데이터의 인수를 웹 브라우저의 URL로 전송함
- Request URI로 식별된 리소스를 가져올 수 있도록 요구



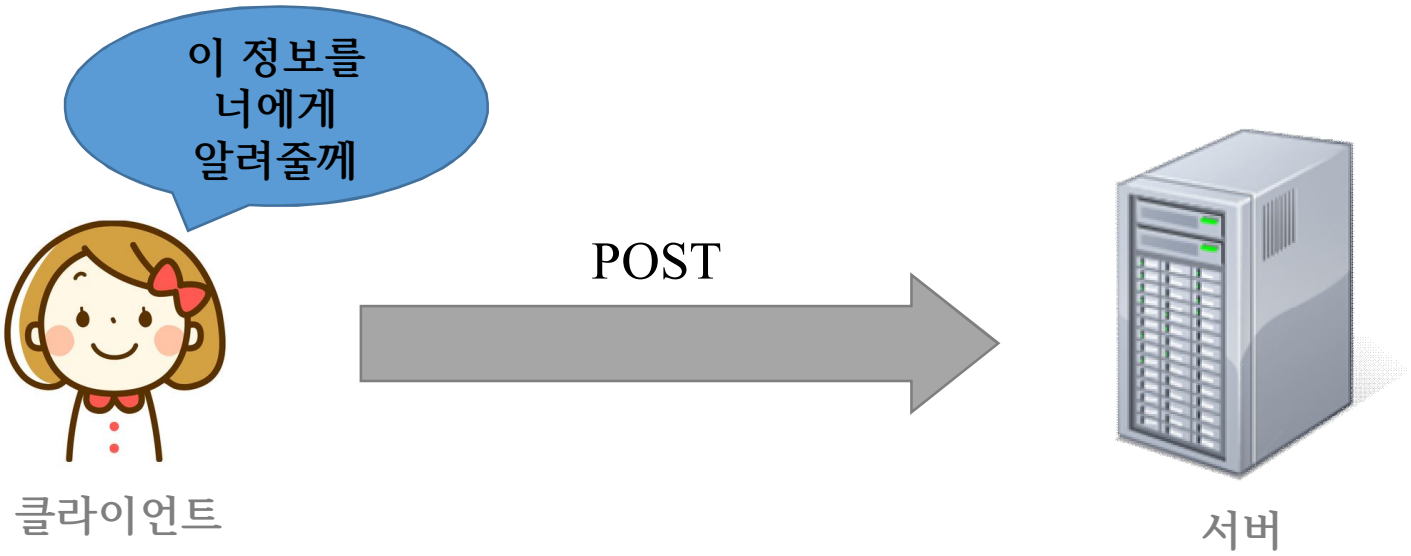
HTTP Request : Get 방식

- 데이터가 주소 입력란에 표시되므로 최소한의 보안도 유지되지 않는 취약한 방식



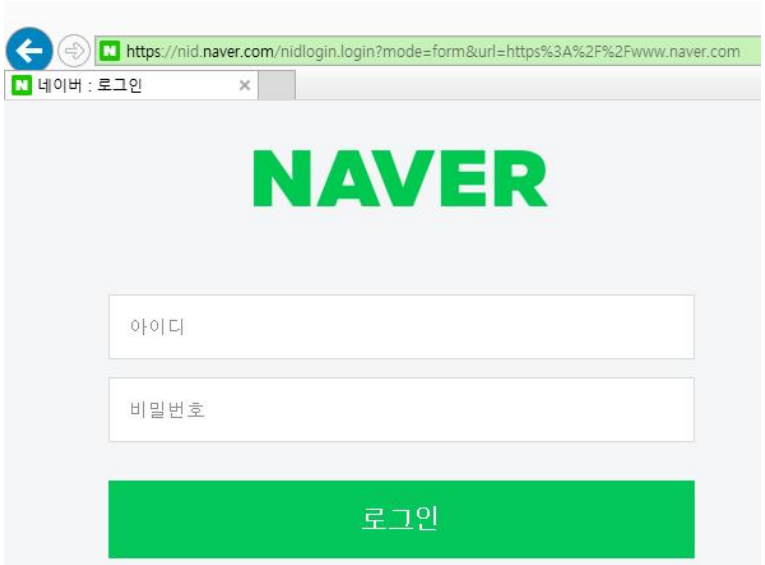
HTTP Request : POST 방식

- 엔티티를 전송하기 위해 사용
- GET으로도 엔티티를 전송할 수 있지만, 자주 사용하지 않고 일반적으로 POST를 사용



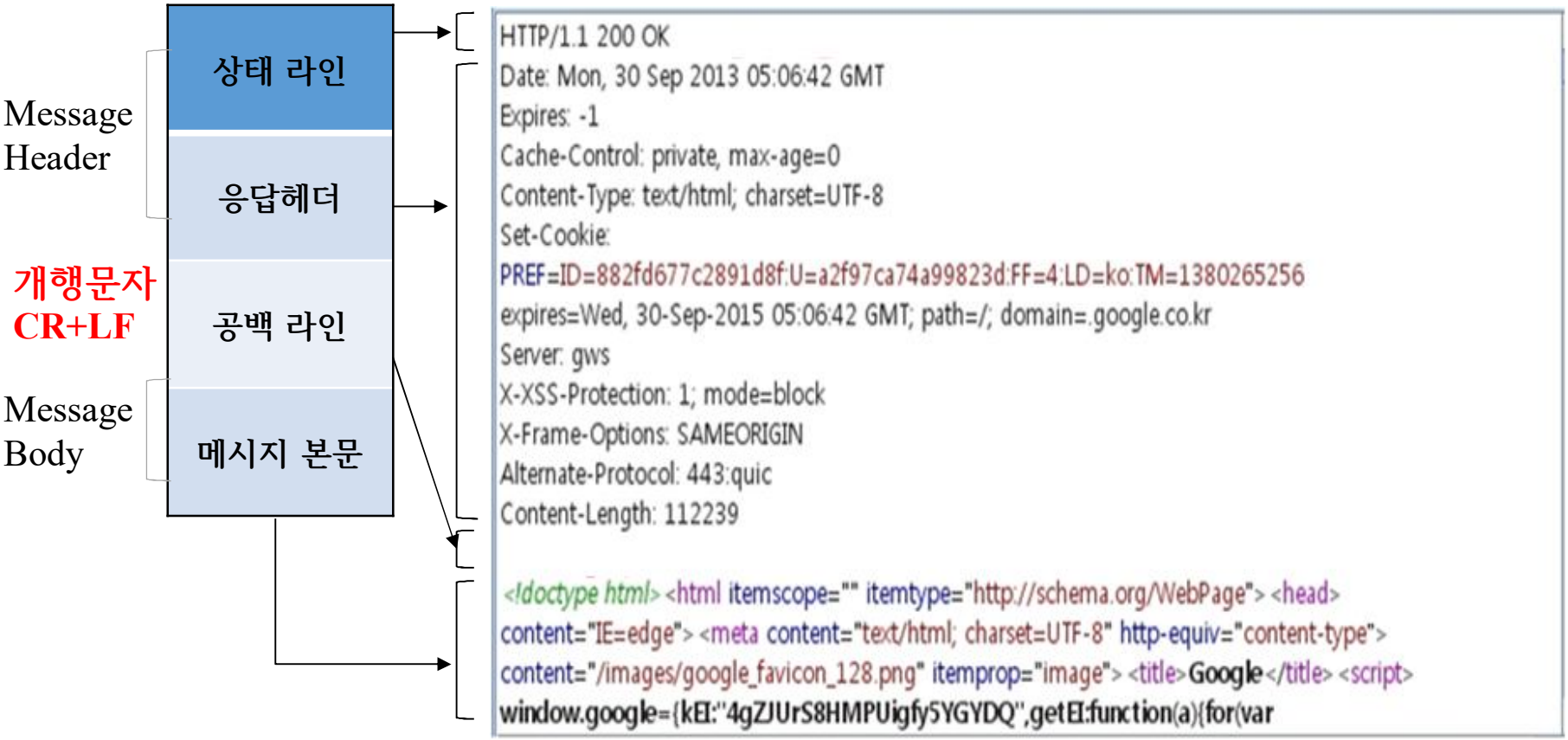
HTTP Request : POST 방식

- URL에 요청 데이터를 기록하지 않고 HTTP 헤더에 데이터를 전송.
- 엔티티를 URL로 전송하지 않으므로 다른 사용자가 링크로 해당 페이지를 볼 수 없음



게시판의 경우, 목록이나 글 보기 화면은 접근 자유도를 위해 GET 방식을 사용, 게시글을 저장·수정·삭제하거나 대용량 데이터를 전송할 때는 POST 방식을 사용.

▪ HTTP Response Message



- 헤더(Request headers)

Header	Description
Accept-Range	바이트 단위의 요구를 수신할 수 있는지 없는지 여부
Age	리소스의 지정 경과 시간
Etag	리소스 특정하기 위한 정보
Location	클라이언트를 지정한 URI에 리다이렉트
Proxy-Authenticate	프록시 서버의 클라이언트 인증을 위한 정보
Retry-After	리퀘스트 재시행의 타이밍 정보
Server	HTTP 서버 정보
Vary	프록시 서버에 대한 캐시 관리 정보
WWW-Authenticate	서버의 클라이언트 인증을 위한 정보

HTTP Response 코드

실행 결과 코드	내용	설명
100번대	정보 전송	HTTP 1.0까지는 계열에 대한 정의가 이루어지지 않았기 때문에 실험 용도 외에는 100번대 서버 측의 응답이 없다.
200번대	성공	클라이언트의 요구가 성공적으로 수신 및 처리되었음을 의미한다.
300번대	리다이렉션	해당 요구 사항을 처리하기 위해 사용자 에이전트가 수행해야 할 추가적인 동작이 있음을 의미한다.
400번대	클라이언트 측 에러	클라이언트에 오류가 발생했을 때 사용한다. 예를 들면 클라이언트가 서버에 보내는 요구 메시지를 완전히 처리하지 못한 경우 등이다.
500번대	서버 측 에러	서버 자체에서 발생한 오류 상황이나 요구 사항을 제대로 처리할 수 없을 때 사용한다.